

ДИСЦИПЛИНА	Базы данных и анализ промышленных данных
ИНСТИТУТ	Передовая инженерная школа
КАФЕДРА	Кафедра передовых технологий
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Астафьев Р.У.
СЕМЕСТР	2 семестр, 2025 - 2026 гг.

Практическая работа 9

Классификация: методы предпосылки, методы проведения, анализ результатов.

Классификация – это процесс разбиения множества объектов на отдельные классы по известным свойствам.

Например, фильтрация спама на почте, распределение новостей по тематикам, распознавание изображений.

Основные методы классификации:

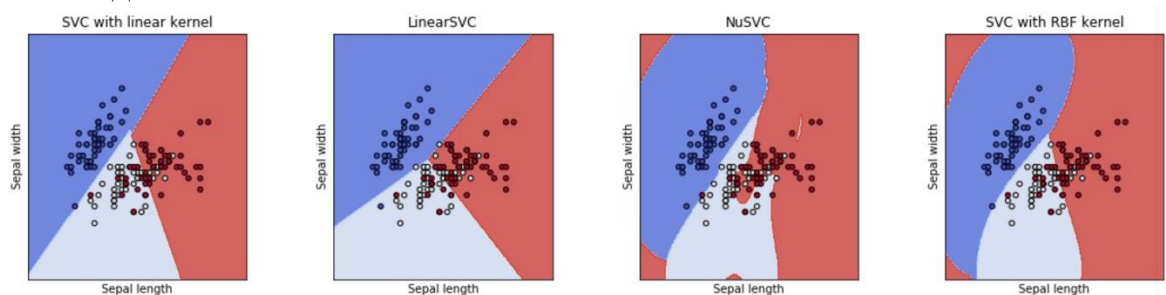
1. Логистическая регрессия — модель, которая предсказывает вероятность принадлежности объекта к определённому классу на основе линейной комбинации признаков.

2. Деревья решений — метод, который представляет собой иерархическую структуру, где каждый узел принимает решение о разделении данных на основе их признаков.

3. Метод k-ближайших соседей (k-NN) — алгоритм, который относит объект к классу на основе голосования его k ближайших соседей в пространстве признаков.

4. Нейронные сети — модели, состоящие из слоёв искусственных нейронов, которые обучаются на основе примеров и способны находить сложные зависимости между признаками.

5. Метод опорных векторов (SVM) — алгоритм, который ищет оптимальную разделяющую гиперплоскость между классами, максимизируя расстояние до ближайших точек.



```
# Рассмотрим пример классификации
df=pd.read_csv('final_test.csv') # Загружаем датасет о размерах одежды
# Подробное описание данных:
https://www.kaggle.com/datasets/tourist55/clothesizeprediction/data
df.head() # Проверяем загрузку датасета
```

	weight	age	height	size
0	62	28.0	172.72	XL
1	59	36.0	167.64	L
2	61	34.0	165.10	M
3	65	27.0	175.26	L
4	62	45.0	172.72	M

Удалим пропущенные значения

```
df.isna().sum()
```

```
weight    0
age       257
height    330
size      0
dtype: int64
```

```
df['age'] = df['age'].fillna(df['age'].median())
```

```
df['height'] = df['height'].fillna(df['height'].median())
```

Также для повышения точности работы алгоритмов добавим новые переменные

```
df["bmi"] = df["height"]/df["weight"]
```

```
df["weight-squared"] = df["weight"] * df["weight"]
```

Разделим выборку на обучающую и тестовую. На обучающей выборке будет производиться тренировка модели, на тестовой – проверка точности

from sklearn.model_selection **import** train_test_split # Для разделения используем функцию train_test_split

```
X = df.drop('size', axis=1)
```

```
y = df['size']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
random_state = 42) # test_size задает соотношение разделения, в данном
случае 80 на 20, random_state позволяет сохранять разбиение при новом
обучении модели
```

Итоговое разделение

```
len(X_train), len(X_test)
```

```
(95787, 23947)
```

from sklearn.ensemble **import** RandomForestClassifier # Для классификации будем использовать модель случайного леса

from sklearn.metrics **import** accuracy_score # Импортируем метрику точности

```
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
```

Создаем объект с настройками

```
rf_classifier.fit(X_train, y_train) # Обучение модели
```

```
y_pred = rf_classifier.predict(X_test) # Прогнозируем классы для тестовой
выборки
```

```
accuracy = accuracy_score(y_test, y_pred) # Оцениваем точность модели
```

```
print(f"Точность модели: {accuracy:.2f}") # Точность модели назкая,
примерно в половине случаев классификатор будет работать правильно
```

```
Точность модели: 0.51
```

Попробуем улучшить точность при помощи GridSearch – функции, которая перебирает различные настройки модели и возвращает наилучший результат

from sklearn.model_selection **import** GridSearchCV

```
param_grid = { # Задаем параметры настроек для перебора
```

```
'n_estimators': [50, 100, 200],
```

```

    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
grid_search = GridSearchCV(estimator=rf_classifier,
param_grid=param_grid, cv=3, n_jobs=-1, verbose=2) # Создание экземпляра
GridSearchCV
grid_search.fit(X_train, y_train) # Поиск лучших параметров

# Выводим лучшие параметры модели
print(f"Лучшие параметры: {grid_search.best_params_}")
    Лучшие параметры: {'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}

best_rf_classifier = grid_search.best_estimator_ # Передаем лучшие
параметры для предсказания
y_pred_best = best_rf_classifier.predict(X_test) # Предсказания на
тестовой выборке
best_accuracy = accuracy_score(y_test, y_pred_best) # Оценка точности
модели с лучшими параметрами
print(f"Точность модели с лучшими параметрами: {best_accuracy:.2f}")
# С помощью GridSearch получилось увеличить точность модели на 0,01
# Также точность можно увеличить при помощи другой модели, новых
переменных, новому подходу к обработке, предварительного проведения
факторного анализа – т.е. процесс с момента обработки данных до обучения
модели является полностью экспериментальным и заранее невозможно
предугадать какая модель отработает наилучшим образом.

    Точность модели с лучшими параметрами: 0.52

```

Задание 1 (4 балла)

1. Скачать данные по ссылке

<https://www.kaggle.com/datasets/nikhil7280/weather-type-classification/data>

2. Построить модели классификации данных (y – Weather Type) используя от 3-х различных алгоритмов (например, k-NN, SVM и т.д.);
3. Самостоятельно изучить другие метрики качества для моделей (Precision, Recall, F1-мера и тд);
4. Сравнить качество полученных моделей.

Задание * (4 балла)

1. Для данных из примера

(<https://www.kaggle.com/datasets/tourist55/clothesizeprediction/data>) улучшить точность модели классификации до 0,7 и выше.

Результаты представить в формате .ipynb.